

Bases de Datos Distribuidas

Guia de estudio Fragmentacion — salesdb

Fragmentacion Vertical | Fragmentacion Horizontal

1. Que es Fragmentar

Fragmentar = dividir la base de datos original en bases mas pequenas llamadas fragmentos, cada una con un subconjunto de los datos.



Fragmentacion VERTICAL

Divide por COLUMNAS (tablas). Cada fragmento tiene una o mas tablas completas, sin mezclar con otras.



Fragmentacion HORIZONTAL

Divide por FILAS. Cada fragmento tiene todas las columnas pero solo ciertos registros (filtrado con WHERE).

Flujo general — siempre el mismo orden:

1. Crear la nueva base de datos (CREATE DATABASE)
2. Crear las tablas con su estructura (CREATE TABLE) — primero tablas PADRE, luego HIJO
3. Copiar datos desde salesdb con INSERT ... SELECT
4. Verificar con SELECT COUNT(*)

2. Esquema de los Fragmentos

```
salesdb  (base original)
|
|---> customerDB  [VERTICAL]  tablas: address + customer
|---> supplierDB  [VERTICAL]  tablas: address + supplier
|---> zona1DB     [HORIZONTAL] CDMX e Hidalgo
|---> zona2DB     [HORIZONTAL] Queretaro y Puebla
+---> zona3DB     [HORIZONTAL] Veracruz y Morelos
```

3. Fragmentos Verticales

3.1 customerDB

Contiene: tablas address y customer completas, sin filtrar filas.

Paso 1 — Crear la base de datos

```
CREATE DATABASE IF NOT EXISTS customerDB
  DEFAULT CHARACTER SET utf8;
```

Paso 2 — Crear tablas (orden: address primero, luego customer)

```
USE customerDB;

CREATE TABLE address (
  addressID INT(11) NOT NULL AUTO_INCREMENT,
  street VARCHAR(100) DEFAULT NULL,
  localy VARCHAR(50) DEFAULT NULL,
  city VARCHAR(50) DEFAULT NULL,
  postcode VARCHAR(10) DEFAULT NULL,
  state VARCHAR(50) DEFAULT NULL,
  PRIMARY KEY (addressID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

CREATE TABLE customer (
  customerID INT(11) NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) DEFAULT NULL,
  phone VARCHAR(20) DEFAULT NULL,
  email VARCHAR(100) DEFAULT NULL,
  addressID INT(11) DEFAULT NULL,
  PRIMARY KEY (customerID),
  KEY addressID (addressID),
  CONSTRAINT customer_ibfk_1
    FOREIGN KEY (addressID) REFERENCES address (addressID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Paso 3 — Copiar datos (CORRECCION: insertar TODAS las direcciones)

Por que insertar TODAS las direcciones y no solo las de clientes?

Algunos clientes tienen addressID que apunta a direcciones que no tienen otro cliente asociado. Si filtramos con INNER JOIN, esas direcciones no entran y MySQL lanza Error 1452 (FK fail). Solucion: en fragmentos VERTICALES, insertar address sin filtro.

```
-- Si algo quedo a medias, limpiar primero:
```

```

TRUNCATE customerDB.customer;
TRUNCATE customerDB.address;

-- 1. TODAS las direcciones (sin filtro - leccion aprendida)
INSERT INTO customerDB.address
SELECT * FROM salesdb.address;

-- 2. Todos los clientes
INSERT INTO customerDB.customer
SELECT * FROM salesdb.customer;

```

Paso 4 — Verificar

```

SELECT 'address' AS tabla, COUNT(*) AS registros FROM customerDB.address
UNION ALL
SELECT 'customer', COUNT(*) FROM customerDB.customer;

-- Resultado esperado:
-- address | 100
-- customer | 100

```

3.2 supplierDB

Contiene: tablas address y supplier completas, sin filtrar filas.

Script completo (ejecutar todo de una vez)

```

-- 1. Crear BD
CREATE DATABASE IF NOT EXISTS supplierDB DEFAULT CHARACTER SET utf8;
USE supplierDB;

-- 2. Crear tablas
CREATE TABLE address (
  addressID INT(11) NOT NULL AUTO_INCREMENT,
  street VARCHAR(100) DEFAULT NULL,
  localy VARCHAR(50) DEFAULT NULL,
  city VARCHAR(50) DEFAULT NULL,
  postcode VARCHAR(10) DEFAULT NULL,
  state VARCHAR(50) DEFAULT NULL,
  PRIMARY KEY (addressID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

CREATE TABLE supplier (
  supplierID INT(11) NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) DEFAULT NULL,
  phone VARCHAR(20) DEFAULT NULL,
  email VARCHAR(100) DEFAULT NULL,
  addressID INT(11) DEFAULT NULL,
  PRIMARY KEY (supplierID),
  KEY addressID (addressID),
  CONSTRAINT supplier_ibfk_1
  FOREIGN KEY (addressID) REFERENCES address (addressID)
)

```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

-- 3. Copiar datos
INSERT INTO supplierDB.address
SELECT * FROM salesdb.address;

INSERT INTO supplierDB.supplier
SELECT * FROM salesdb.supplier;

-- 4. Verificar
SELECT 'address' AS tabla, COUNT(*) AS registros FROM supplierDB.address
UNION ALL
SELECT 'supplier', COUNT(*) FROM supplierDB.supplier;
```

4. Fragmentos Horizontales

En fragmentos horizontales el filtro es consistente: solo entran los clientes del estado Y solo entran las direcciones de ese estado, por eso el INNER JOIN + WHERE funciona sin error 1452.

Regla de oro de fragmentacion horizontal:

Todos los addressID que existan en customer deben existir en address DENTRO del mismo fragmento.

Por eso filtramos ambas tablas con el mismo WHERE state IN (...)

4.1 zona1DB — CDMX e Hidalgo

Script completo

```
CREATE DATABASE IF NOT EXISTS zona1DB DEFAULT CHARACTER SET utf8;
USE zona1DB;

CREATE TABLE address (
  addressID INT(11) NOT NULL AUTO_INCREMENT,
  street VARCHAR(100) DEFAULT NULL,
  locally VARCHAR(50) DEFAULT NULL,
  city VARCHAR(50) DEFAULT NULL,
  postcode VARCHAR(10) DEFAULT NULL,
  state VARCHAR(50) DEFAULT NULL,
  PRIMARY KEY (addressID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

CREATE TABLE customer (
  customerID INT(11) NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) DEFAULT NULL,
  phone VARCHAR(20) DEFAULT NULL,
  email VARCHAR(100) DEFAULT NULL,
  addressID INT(11) DEFAULT NULL,
  PRIMARY KEY (customerID),
  KEY addressID (addressID),
  CONSTRAINT customer_ibfk_1
    FOREIGN KEY (addressID) REFERENCES address (addressID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

CREATE TABLE customerorder (
  orderID INT(11) NOT NULL AUTO_INCREMENT,
  customerID INT(11) DEFAULT NULL,
  date DATE DEFAULT NULL,
  total DECIMAL(10,2) DEFAULT NULL,
  paymentMethod VARCHAR(50) DEFAULT NULL,
  status VARCHAR(50) DEFAULT NULL,
  PRIMARY KEY (orderID),
  KEY customerID (customerID),
  CONSTRAINT customerorder_ibfk_1
    FOREIGN KEY (customerID) REFERENCES customer (customerID)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```

-- Copiar datos filtrados por estado
INSERT INTO zona1DB.address
SELECT * FROM salesdb.address
WHERE state IN ('CDMX', 'Hidalgo');

INSERT INTO zona1DB.customer
SELECT c.*
FROM salesdb.customer c
INNER JOIN salesdb.address a ON c.addressID = a.addressID
WHERE a.state IN ('CDMX', 'Hidalgo');

INSERT INTO zona1DB.customerorder
SELECT co.*
FROM salesdb.customerorder co
INNER JOIN salesdb.customer c ON co.customerID = c.customerID
INNER JOIN salesdb.address a ON c.addressID = a.addressID
WHERE a.state IN ('CDMX', 'Hidalgo');

-- Verificar
SELECT 'address' AS tabla, COUNT(*) AS registros FROM zona1DB.address
UNION ALL
SELECT 'customer', COUNT(*) FROM zona1DB.customer
UNION ALL
SELECT 'customerorder', COUNT(*) FROM zona1DB.customerorder;

```

4.2 zona2DB — Queretaro y Puebla

Mismo script que zona1DB, cambiando nombre de BD y estados en el WHERE.

```

CREATE DATABASE IF NOT EXISTS zona2DB DEFAULT CHARACTER SET utf8;
USE zona2DB;

-- (mismos CREATE TABLE que zona1DB)
-- ... address, customer, customerorder ...

-- Solo cambia el WHERE:
INSERT INTO zona2DB.address
SELECT * FROM salesdb.address
WHERE state IN ('Queretaro', 'Puebla');

INSERT INTO zona2DB.customer
SELECT c.*
FROM salesdb.customer c
INNER JOIN salesdb.address a ON c.addressID = a.addressID
WHERE a.state IN ('Queretaro', 'Puebla');

INSERT INTO zona2DB.customerorder
SELECT co.*
FROM salesdb.customerorder co
INNER JOIN salesdb.customer c ON co.customerID = c.customerID
INNER JOIN salesdb.address a ON c.addressID = a.addressID
WHERE a.state IN ('Queretaro', 'Puebla');

```

```
-- Verificar
SELECT 'address'          AS tabla, COUNT(*) FROM zona2DB.address
UNION ALL
SELECT 'customer',        COUNT(*) FROM zona2DB.customer
UNION ALL
SELECT 'customerorder', COUNT(*) FROM zona2DB.customerorder;
```

4.3 zona3DB — Veracruz y Morelos

```
CREATE DATABASE IF NOT EXISTS zona3DB DEFAULT CHARACTER SET utf8;
USE zona3DB;

-- (mismos CREATE TABLE que zona1DB)

INSERT INTO zona3DB.address
SELECT * FROM salesdb.address
WHERE state IN ('Veracruz', 'Morelos');

INSERT INTO zona3DB.customer
SELECT c.*
FROM salesdb.customer c
INNER JOIN salesdb.address a ON c.addressID = a.addressID
WHERE a.state IN ('Veracruz', 'Morelos');

INSERT INTO zona3DB.customerorder
SELECT co.*
FROM salesdb.customerorder co
INNER JOIN salesdb.customer c ON co.customerID = c.customerID
INNER JOIN salesdb.address a ON c.addressID = a.addressID
WHERE a.state IN ('Veracruz', 'Morelos');

-- Verificar
SELECT 'address'          AS tabla, COUNT(*) FROM zona3DB.address
UNION ALL
SELECT 'customer',        COUNT(*) FROM zona3DB.customer
UNION ALL
SELECT 'customerorder', COUNT(*) FROM zona3DB.customerorder;
```

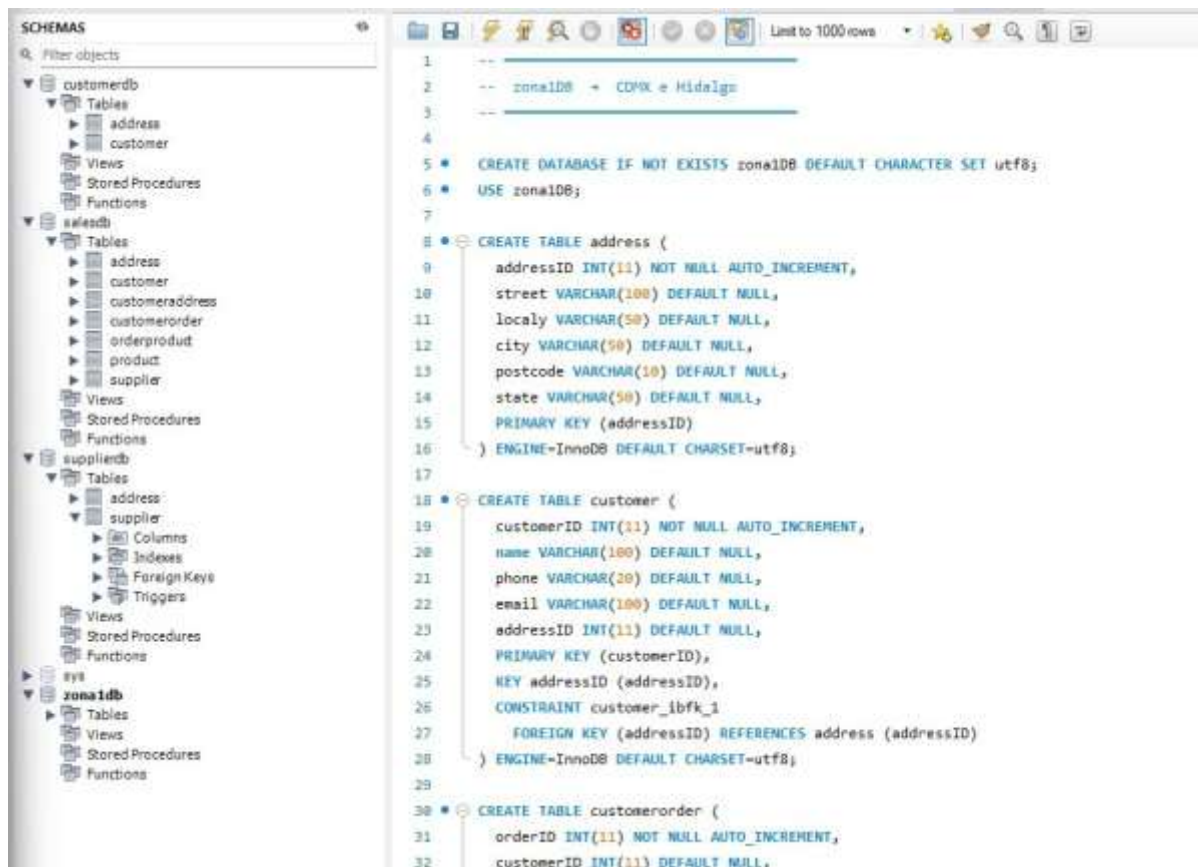
5. Errores Comunes y Soluciones

Error	Por que pasa	Solucion
Cannot add foreign key	Insertas tabla hijo antes que padre	Orden: address → customer → customerorder
0 rows inserted	El WHERE no coincide	Verifica mayusculas: 'CDMX' != 'cdmx'
Duplicate entry	Ejecutaste INSERT dos veces	Ejecuta TRUNCATE tabla; primero
Table doesn't exist	Olvidaste CREATE TABLE o USE	Refresca schemas y verifica el nombre
Error Code: 1452	FK apunta a ID que no existe	Inserta primero TODAS las direcciones

6. Como queda Workbench despues de fragmentar

En el panel izquierdo (Schemas) debes ver las 5 bases de datos creadas:

```
SCHEMAS
customerdb  <-- fragmento vertical (address + customer)
salesdb     <-- base original intacta
supplierdb  <-- fragmento vertical (address + supplier)
sys         <-- sistema, no tocar
zona1db     <-- fragmento horizontal CDMX + Hidalgo
zona2db     <-- fragmento horizontal Queretaro + Puebla (siguiente paso)
zona3db     <-- fragmento horizontal Veracruz + Morelos (siguiente paso)
```

Captura real de MySQL Workbench con los fragmentos creados

7. Hoja de Referencia Rapida para el Examen



Fragmentacion VERTICAL — preguntate:

?Solo quiero ciertas tablas sin filtrar filas? -> Inserta TODAS las direcciones (sin WHERE en address).



Fragmentacion HORIZONTAL — preguntate:

?Quiero filtrar filas por alguna condicion? -> Usa WHERE consistente en TODAS las tablas relacionadas.



Orden de insercion — SIEMPRE:

1ro address → 2do customer → 3ro customerorder (de padre a hijo)



Verificacion — SIEMPRE al final:

SELECT COUNT(*) de cada tabla. Si ves 0 en alguna, revisa el WHERE o si olvidaste ejecutar el INSERT.

Diferencia clave vertical vs horizontal (para el examen):

VERTICAL: Divides por TABLAS — cada fragmento tiene UNA entidad (solo customer, solo supplier)

HORIZONTAL: Divides por FILAS — cada fragmento tiene TODOS los campos pero ciertos registros

En ambos casos la base de datos original (salesdb) queda INTACTA. Solo lees de ella.